

Discrete Mathematics

CS1112

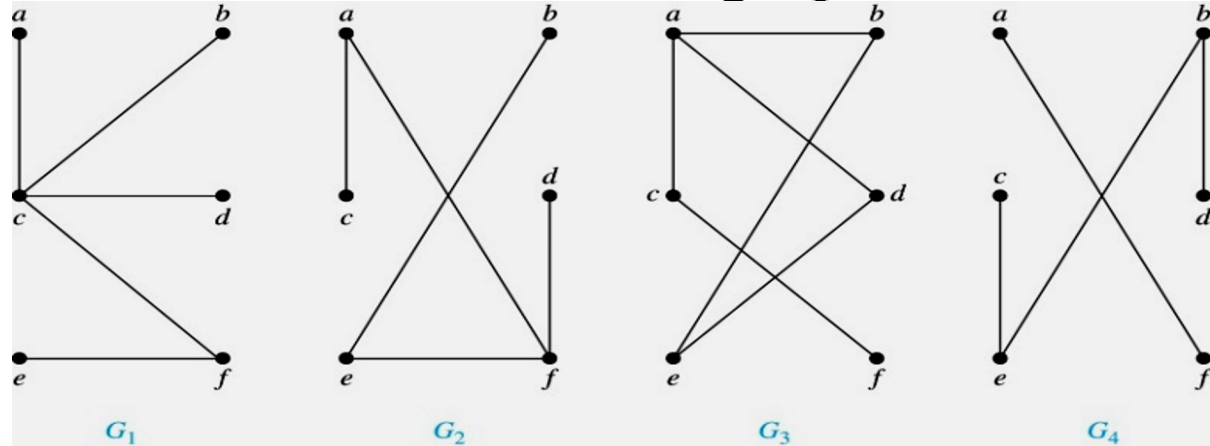
Lecture 5

Trees

Trees

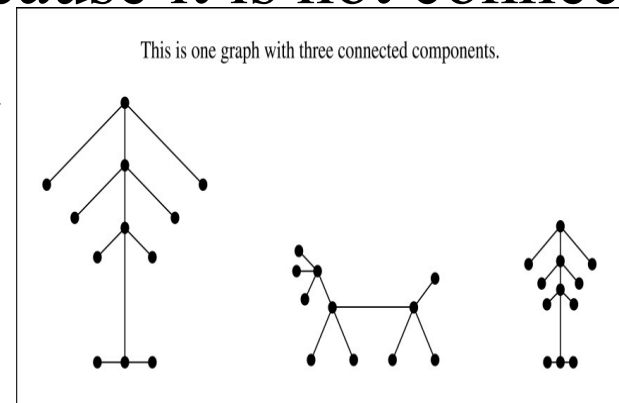
Definition: A *tree* is a connected undirected graph with no simple circuits.

Example: Which of These graphs are trees?



Solution: G_1 and G_2 are trees - both are connected and have no simple circuits. Because e, b, a, d, e is a simple circuit, G_3 is not a tree. G_4 is not a tree because it is not connected.

Definition: A *forest* is a graph that has no simple circuit, but is not connected. Each of the connected components in a forest is a tree



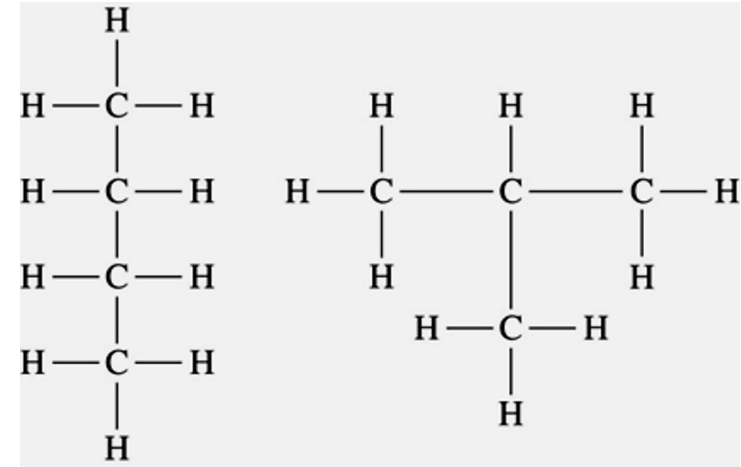
Trees (*continued*)

Theorem: An undirected graph is a tree if and only if there is a unique simple path between any two of its vertices.

Proof:

Trees as Models

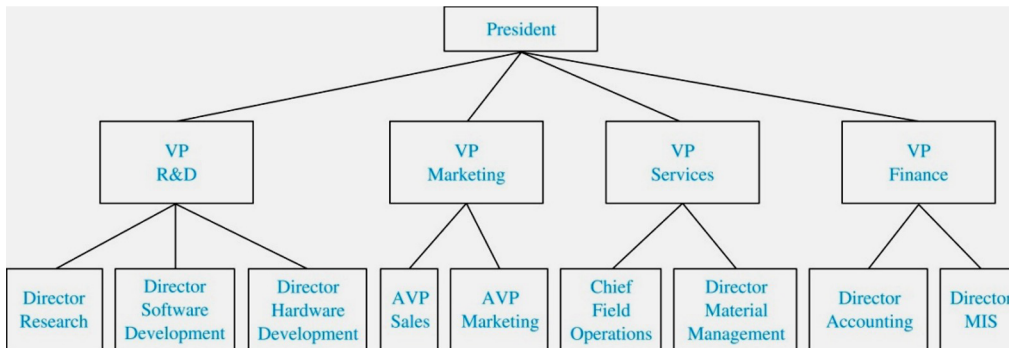
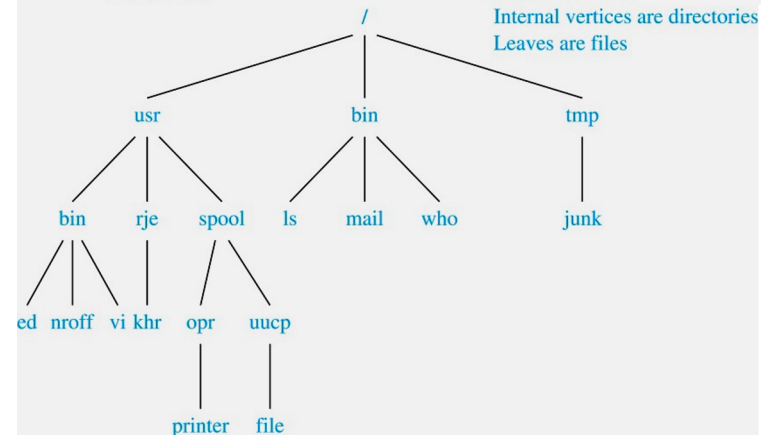
- Trees are used as models in computer science, chemistry, geology, botany, psychology, and many other areas.
- Trees were introduced by the mathematician Cayley in 1857 in his work counting the number of isomers of saturated hydrocarbons. The two isomers of butane are shown at the right.
- The organization of a computer file system into directories, subdirectories, and files is naturally represented as a tree.
- Trees are used to represent the structure of organizations.



Butane

Isobutane

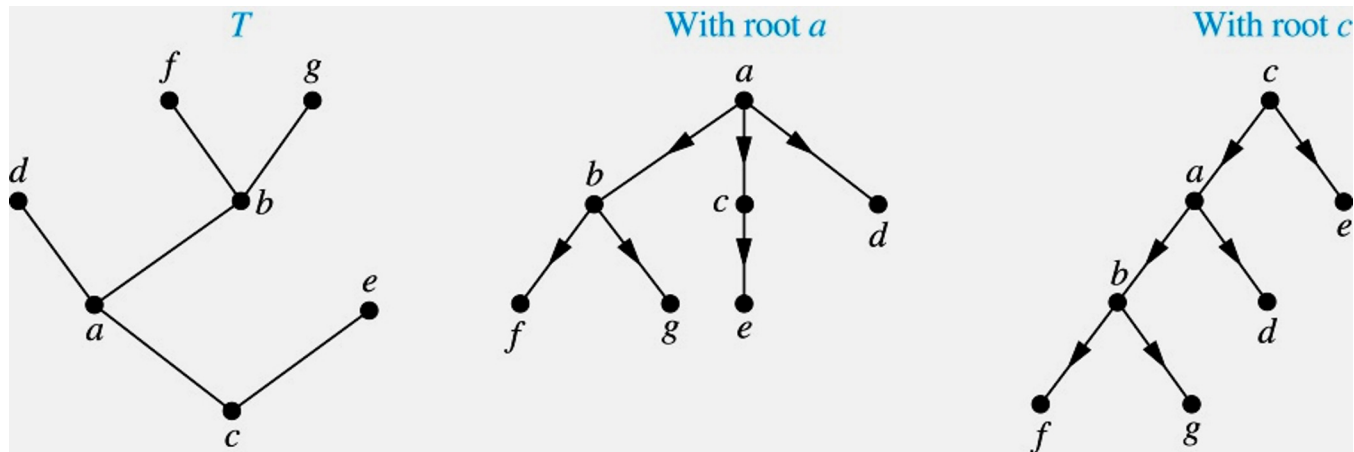
Internal vertices are directories
Leaves are files



Rooted Trees

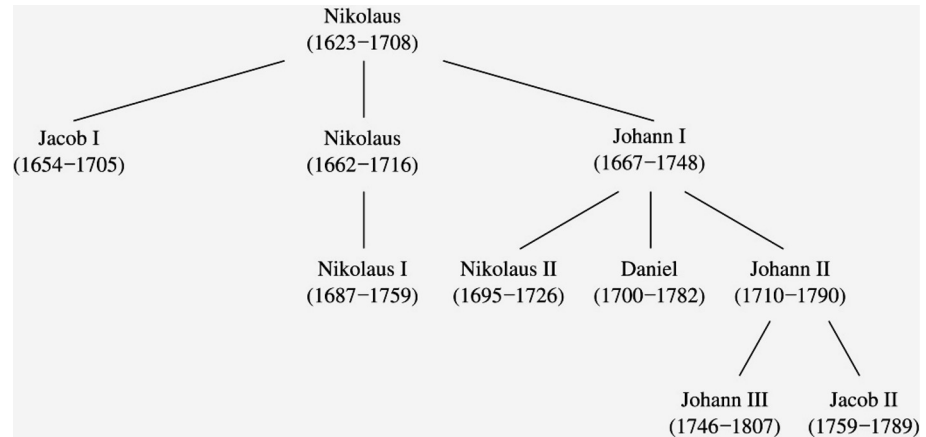
Definition: A *rooted tree* is a tree in which one vertex has been designated as the *root* and every edge is directed away from the root.

An unrooted tree is converted into different rooted trees when different vertices are chosen as the root.



Rooted Tree Terminology

- Terminology for rooted trees is a mix from botany and genealogy (such as this family tree of the Bernoulli family of mathematicians).



- If v is a vertex of a rooted tree other than the root, the *parent* of v is the unique vertex u such that there is a directed edge from u to v . When u is a parent of v , v is called a *child* of u . Vertices with the same parent are called *siblings*.
- The *ancestors* of a vertex are the vertices in the path from the root to this vertex, excluding the vertex itself and including the root. The *descendants* of a vertex v are those vertices that have v as an ancestor.
- A vertex of a rooted tree with no children is called a *leaf*. Vertices that have children are called *internal vertices*.
- If a is a vertex in a tree, the *subtree* with a as its root is the subgraph of the tree consisting of a and its descendants and all edges incident to these descendants.

Basic Terminologies

- Each node may have zero, one or more children.
- Each node has a unique parent, except for the root.

Examples:

Node R has two children

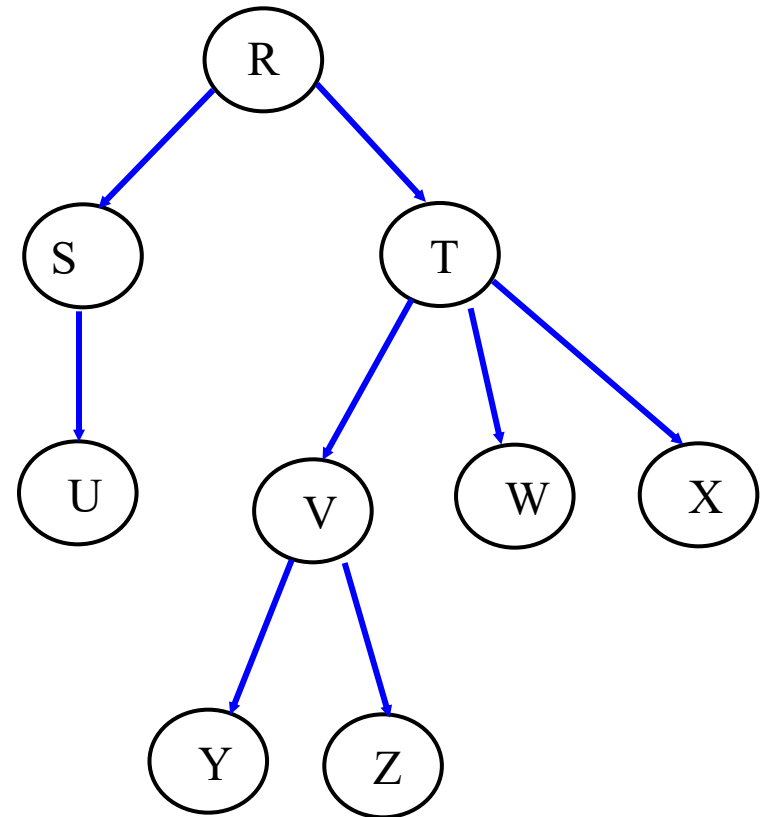
namely nodes S & T

Node S has one child U

Node T has three children

namely nodes V, W & X

Nodes U, Y, Z, W, X have no children



Examples:

Parent of Y is V

Parent of V is T

Parent of T is R

Root node R has no parents

Basic Terminologies

- All children from the same parent are called siblings.
- Nodes without children are known as leaves.
- Nodes with children are known as internal nodes.

Examples:

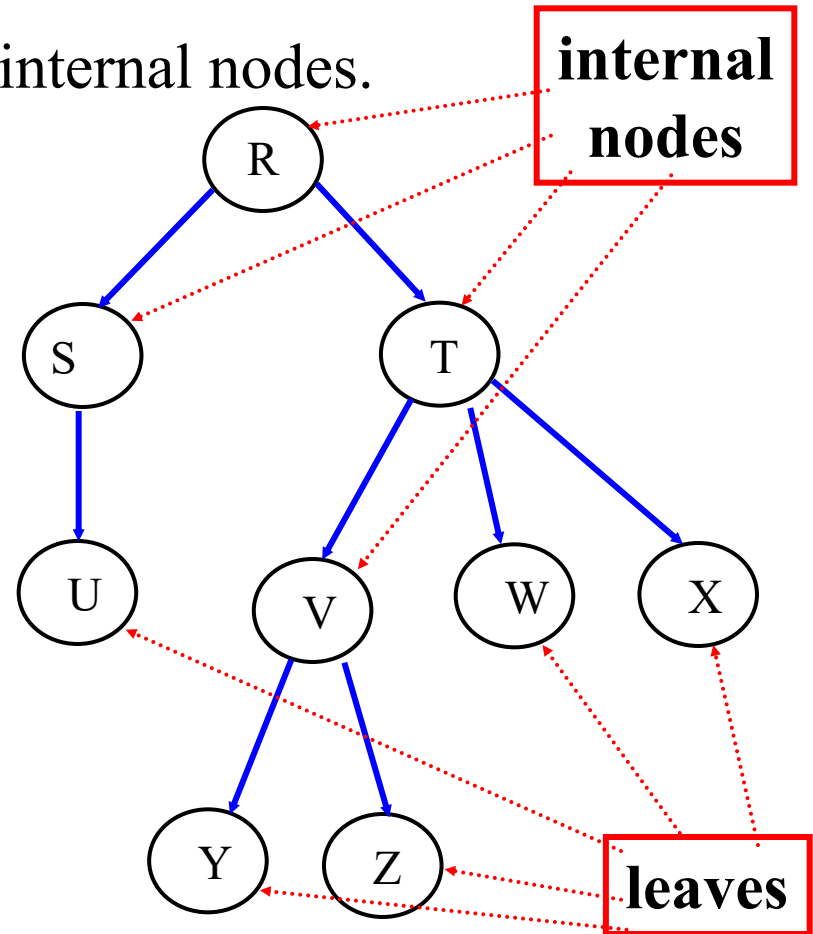
V, W & X are siblings belonging to parent node T

Y, Z are siblings belonging to parent node V

Examples:

U, Y, Z, W, X are leaves

R, S, T, V are internal nodes



Basic Terminologies

- Level (or depth) of a node refers to its distance from the root.
- Height of a tree is its maximum level (or depth) from root.
- Size of a tree refers to the number of nodes present in the tree.

Examples:

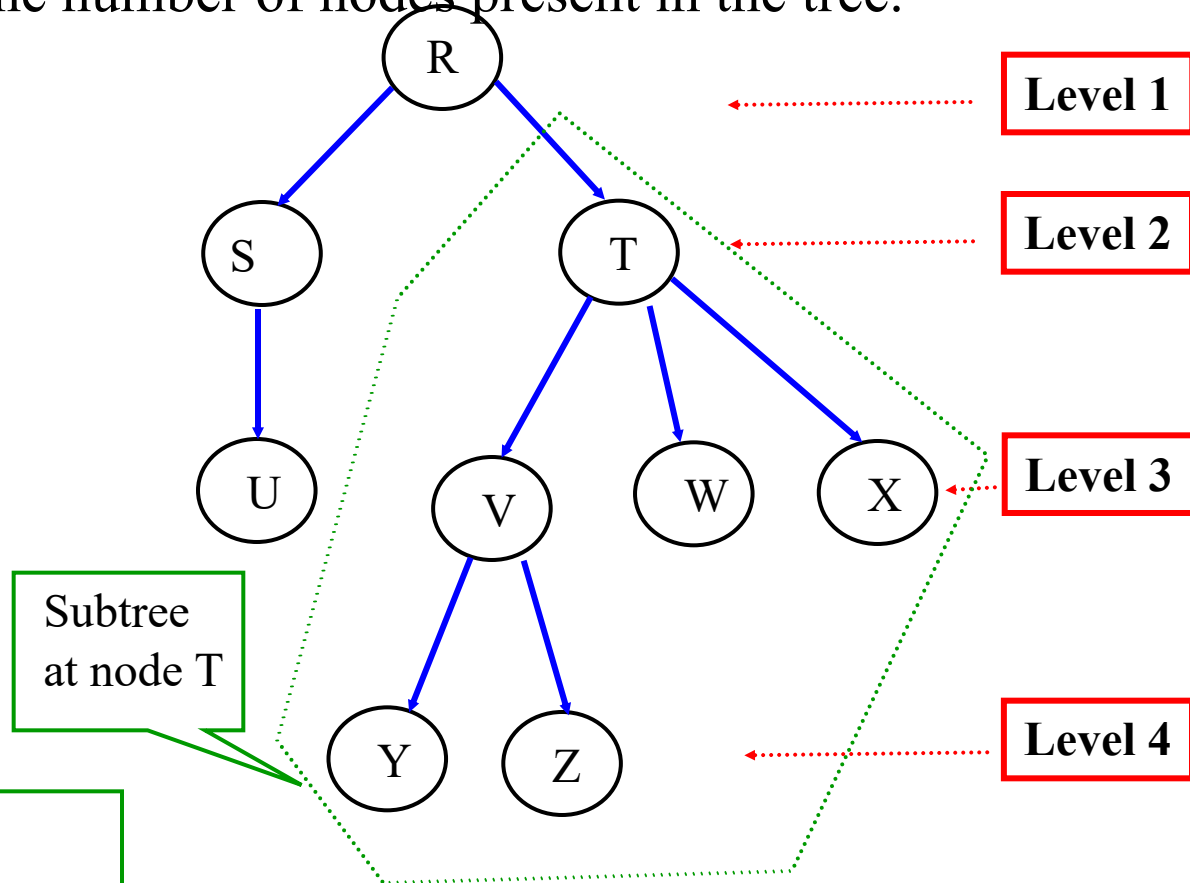
S & T are at Level 2
U, V, W & X are at Level 3
Y & Z are at Level 4

Examples:

Height of Tree
rooted at R is 4
Height of Subtree
at Node T is 3

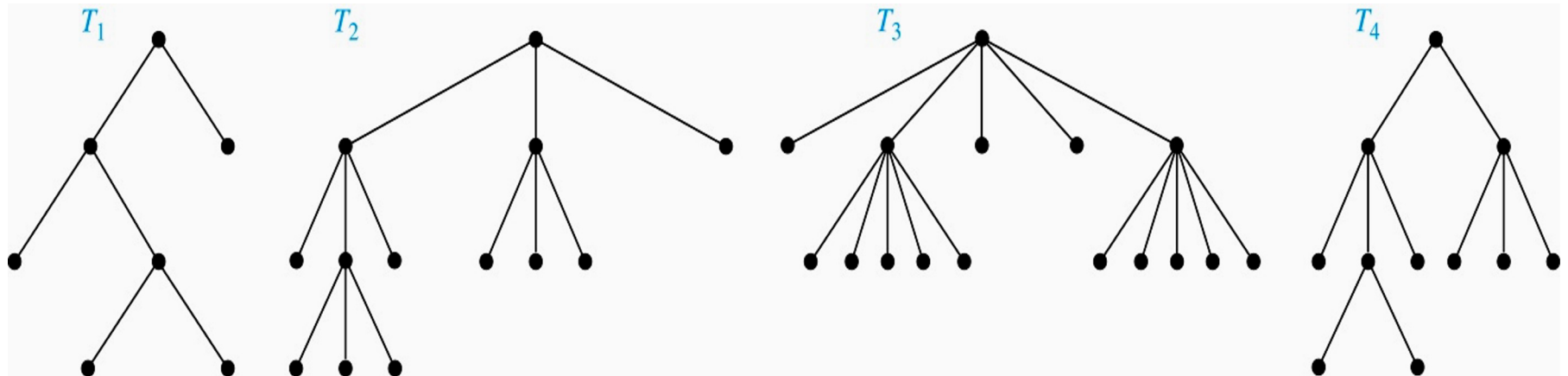
Examples:

Size of Tree at R is 9



m -ary Rooted Trees

Definition: A rooted tree is called an m -ary tree if every internal vertex has no more than m children. An m -ary tree with $m = 2$ is called a *binary* tree.



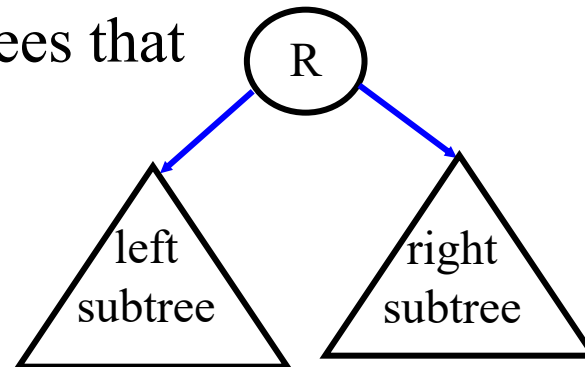
Binary Trees

- A binary tree is a tree in which each node can have at most two children.

Formal Definition:

A binary tree is either

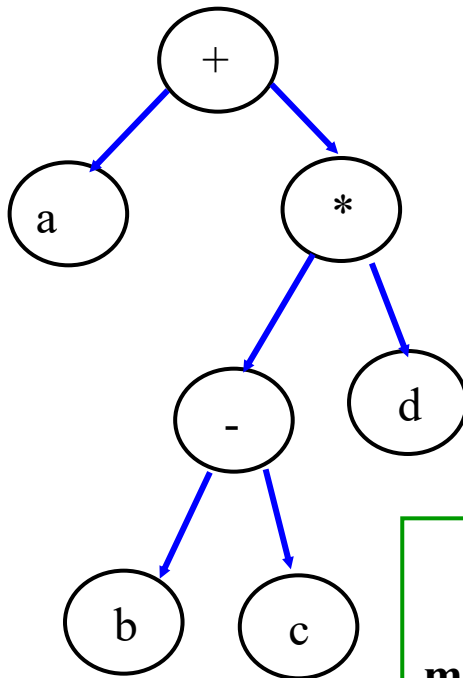
- an empty tree, or
- a node with two subtrees that are also binary trees



Binary Trees

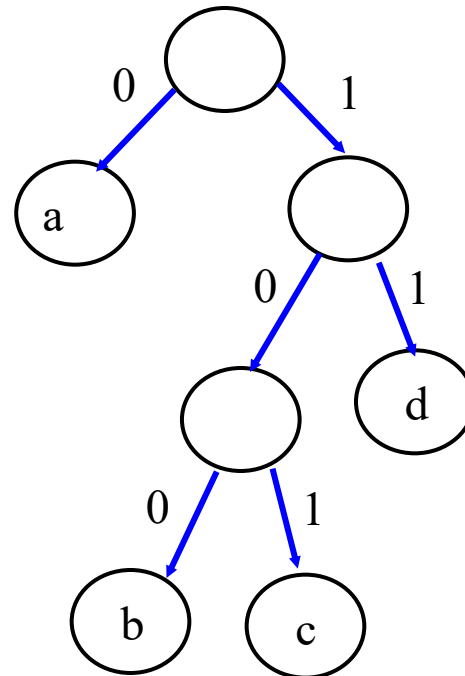
- Binary trees have many important uses. Two examples are:

Expression Tree



denotes
 $a + b - c * d$
more precisely
 $a + ((b - c) * d)$

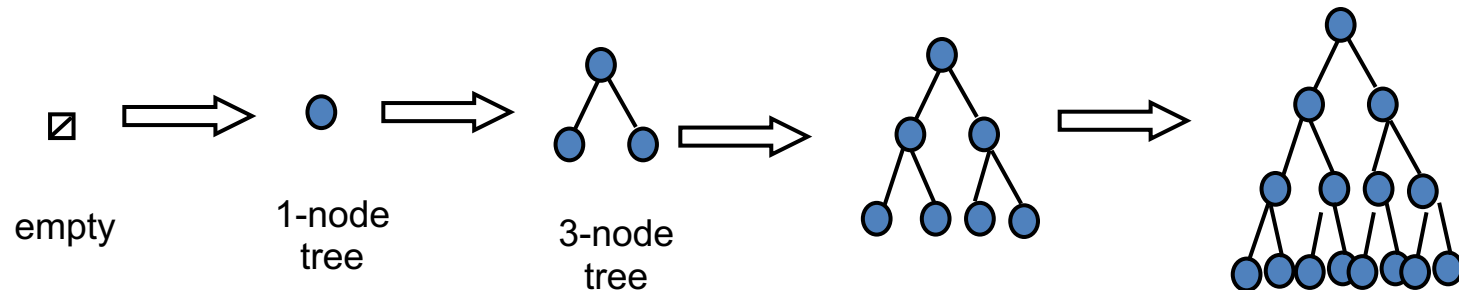
Huffman Coding Tree



denotes
 $a = 0$
 $b = 100$
 $c = 101$
 $d = 11$

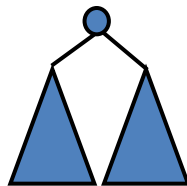
Full Binary Trees

- For full binary trees, the nodes must be added level by level.
- These trees would grow in power of twos minus one (i.e. $2^n - 1$).

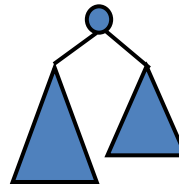


Full Binary Trees

- A binary tree is said to be full if either
 - it is an empty tree, or
 - its two subtrees are full binary trees with the same height



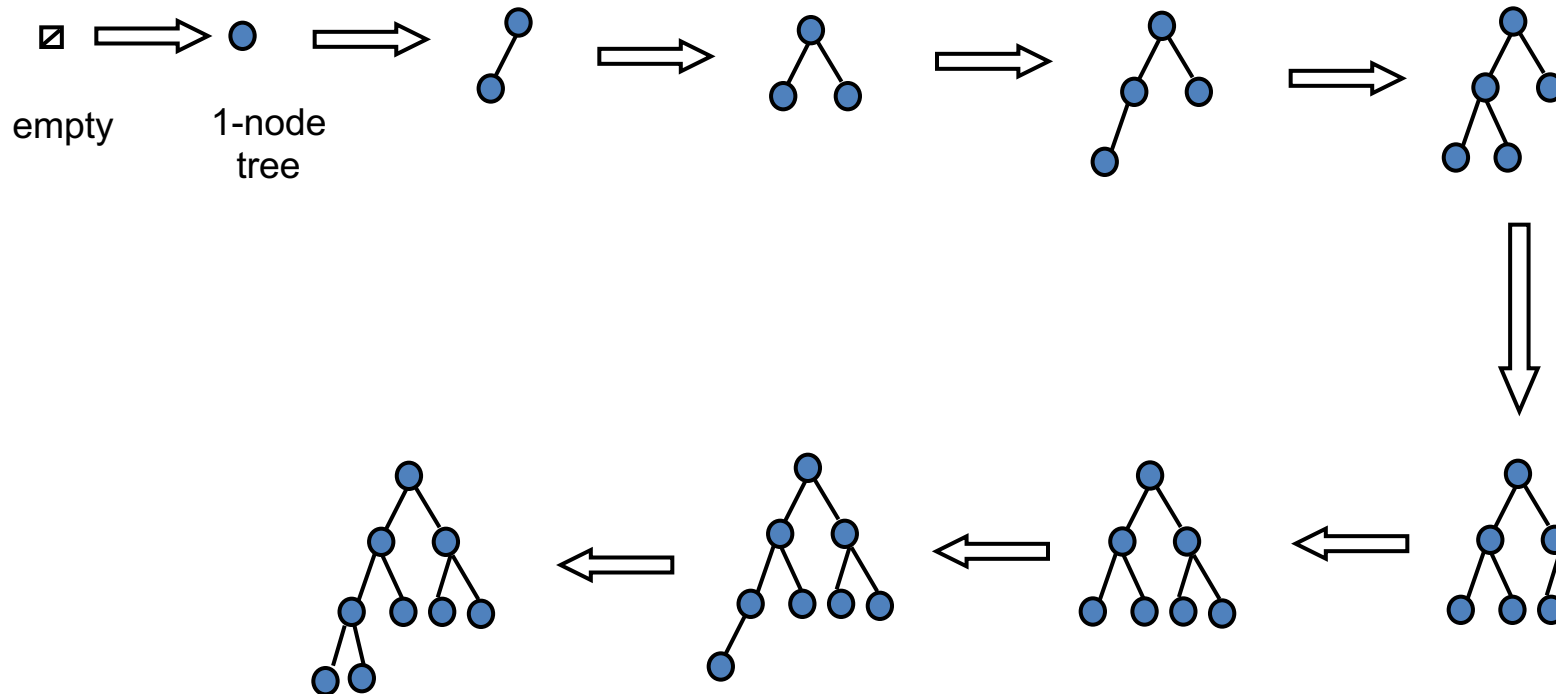
YES!



NO!

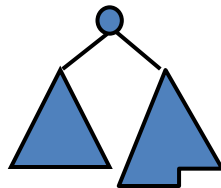
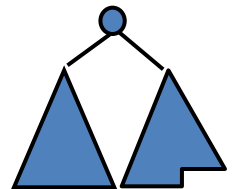
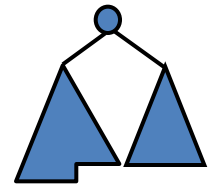
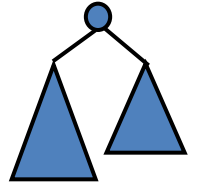
Complete Binary Trees

- Nodes for complete binary tree can be added one node at a time, but must be done level-by-level, in a left to right order.
- For Example:

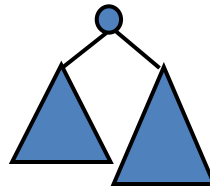


Complete Binary Trees

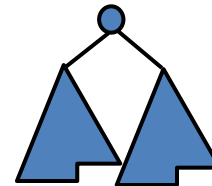
- A binary tree is said to be complete if either
 - it is an empty tree,
 - or both subtrees are full but the left subtree is the same or one deeper than the right; or
 - its left subtree is complete, and right subtree is full but the left subtree is one deeper than the right; or
 - its left subtree is full, and right subtree is complete but both are of the same depth.



NO!



NO!



NO!